Experimentos com um MLP implementado do zero

CI1026 - Visão Computacional e Percepção - TA05 código: https://github.com/AlmiranteFuchs/TA05-mlp

1. Introdução

A tarefa foi estudar o notebook de referência "MLP from scratch" (adaptado de um notebook do Kaggle), ajudar a depurar e descobrir como classificar bem as três classes do Iris, analisando convergência, erros e overfitting.

O Iris tem 150 amostras de 4 features (medidas de sépala e pétala) e 3 classes. A rede do notebook é um 4-5-3 com sigmoid, erro quadrático e SGD por amostra com one-hot na saída.

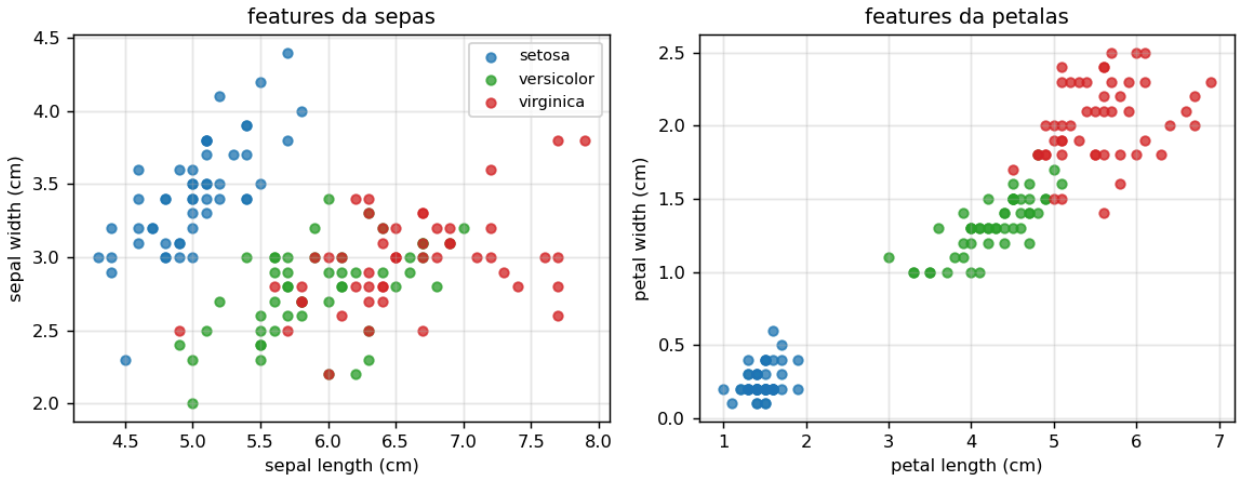


Figura 1: as 3 classes do Iris: sepas (esq.) quase inseparaveis, petalas (dir.) bem melhores

Spoiler do relatório inteiro: a rede aprende direitinho, mas o predict dela joga o aprendizado fora. Treinada por 3000 épocas ela marca 33% de acurácia no teste (nível de chute), enquanto a mesma rede, nos mesmos pesos, com o predict correto marca 100%. Detalhes abaixo.

2. Depuração do código de referência

Portei o notebook fielmente (`src/mlp_ref.py`, bugs preservados) e escrevi uma versão corrigida (`src/mlp.py`). O que encontrei:

- O `predict` não aplica a função de ativação. O forward de treino usa sigmoid nas duas camadas; o predict faz só `X @ W1 + b1 @ W2 + b2`, um forward linear. Como a ativação escondida também some, a função decidida muda de verdade. Resultado: acc de teste 0.333; trocando só o predict pelo forward com ativação, a mesma rede treinada dá 1.000. A rede aprendia, quem reprovava era o predict.
- O bias de saída é atualizado dentro do loop dos neurônios escondidos. `BIAS_output[j] -= lr*delta` fica dentro do `for i in range(hiddenLayer)`, então o bias de saída andava 5x por amostra, sempre com o mesmo update repetido.
- Os biases nascem como arrays de int (`np.array([-1,-1,-1])` é int64). O `b[j] -= 0.4` por elemento trunca pra inteiro em vez de acumular, ou seja, na prática os biases nunca saem de -1 durante o treino. Bug silencioso, não dá erro nenhum.

- O "erro" reportado não é o erro. `total_error += square_error` fica dentro do loop dos 3 neurônios de saída e soma somas parciais (o erro do neurônio 0 conta 3x, o do 1 conta 2x), com fator 0.05 em vez de 0.5. O número que o fit imprime converge pra ~0.005 enquanto o MSE real fica em ~0.021. A curva bonita do notebook passa uma confiança que o modelo não tem.
- `params=None` quebra na cara: o default grava `iasHiddenValue` (typo) e depois lê `self.BiasHiddenValue` -> `AttributeError`.
- Um quirk menor: o erro da camada escondida é propagado com o `WEIGHT_output` já atualizado (o textbook usa os pesos antigos). Converge quase igual na prática, mas não é o gradiente exato.
- E faltam coisas que a análise pede: seed, conjunto de validação e histórico por época.

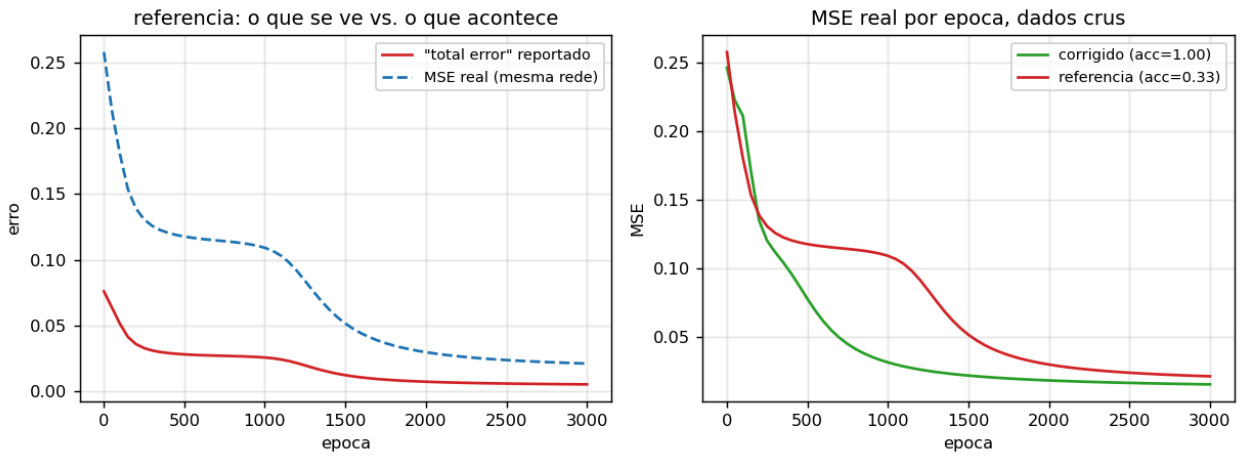


Figura 2: esq.: o 'erro' que o codigo de referencia mostra vs. o MSE real da mesma rede; dir.: MSE real da referencia vs. versao corrigida (dados crus)

À esquerda dá pra ver o erro que o código mostra vs. o MSE real da mesma rede: a curva "bonita" esconde um MSE ~4x maior. À direita, o MSE real da referência vs. a versão corrigida nos mesmos dados crus: a corrigida converge mais cedo (o bias de saída certo e o gradiente exato ajudam) e é ela que entrega 100% de teste, porque o predict agora concorda com o treino.

Um detalhe engraçado: nos outputs salvos do notebook, a execução do próprio professor deu 96.67% (29/30, errou só a única virginica do teste). Ou seja, o estrago do predict depende da sorte da inicialização; na minha seed ele derrubou tudo pra 33%.

3. Setup dos experimentos

Split estratificado 80/20 igual ao do notebook (`random_state=11`): 120 de treino, 30 de teste. Pra análise de overfitting, um split interno de 96 treino / 24 validação. Normalização z-score ajustada só no treino. Todos os sweeps com 3 seeds; o que reporto é média +- desvio no teste.

4. Convergência

Normalização. Com as features cruas (pétala vai de ~1 a 7 cm), a sigmoid trabalha saturada e a descida arrasta: as primeiras ~1500 épocas são gastas só pra destravar. Padronizando as features, a mesma rede com os mesmos pesos iniciais chega a MSE ~0.01 em menos de 400 épocas. A acc final é igual (1.0 nos dois casos, a tarefa é fácil), mas a velocidade muda tudo na prática.

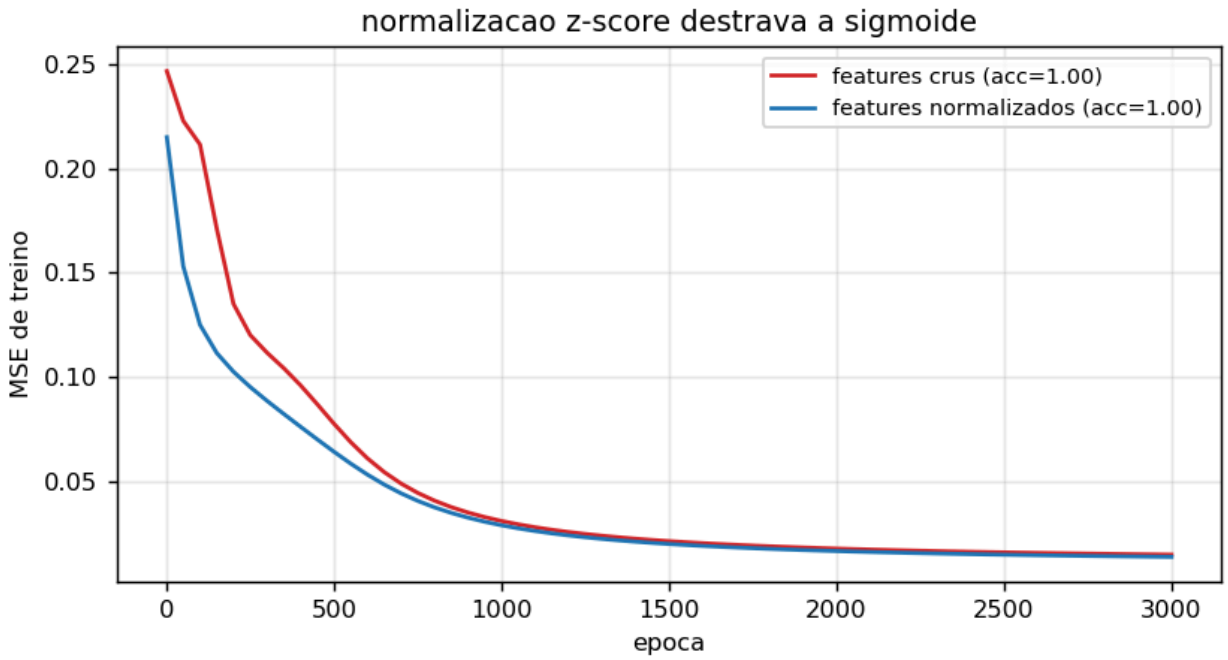


Figura 3: normalizacao z-score: mesma rede, mesmos pesos iniciais, convergencia muito mais rapida

Learning rate. Sweep de 0.0005 a 2.0, rede 4-5-3, sigmoid, dados normalizados, 1200 épocas, 3 seeds:

```
lr=0.0005   acc=0.844 +- 0.031
lr=0.001    acc=0.900 +- 0.072
lr=0.005    acc=1.000 +- 0.000
lr=0.01..2.0  todos 1.000 +- 0.000
```

O lr=0.005 do notebook funciona, mas é conservador: a janela útil aqui vai de ~0.005 a 2.0. MSE + sigmoid em dados normalizados é bem comportado, e nem lr=2.0 diverge (só oscila mais no começo). Abaixo de ~0.005 a rede simplesmente não chega lá em 1200 épocas.

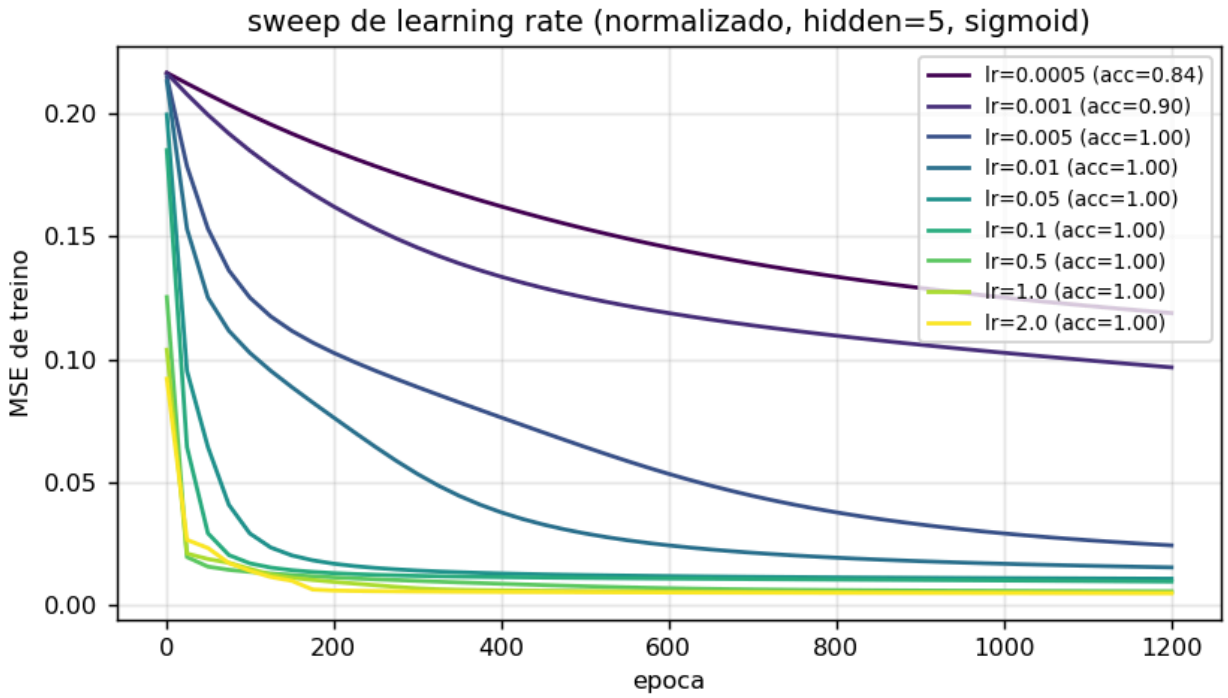


Figura 4: sweep de learning rate: curvas de MSE por epoca (acc de teste na legenda)

5. Capacidade e função de ativação

Neurônios escondidos. Com lr=0.05: H=1 já dá 0.989 +- 0.016; de H=2 pra cima, 1.000 em todas as seeds. Faz sentido, versicolor e virginica são quase separáveis linearmente nas pétalas, então a camada escondida precisa de pouquíssima capacidade pro Iris.

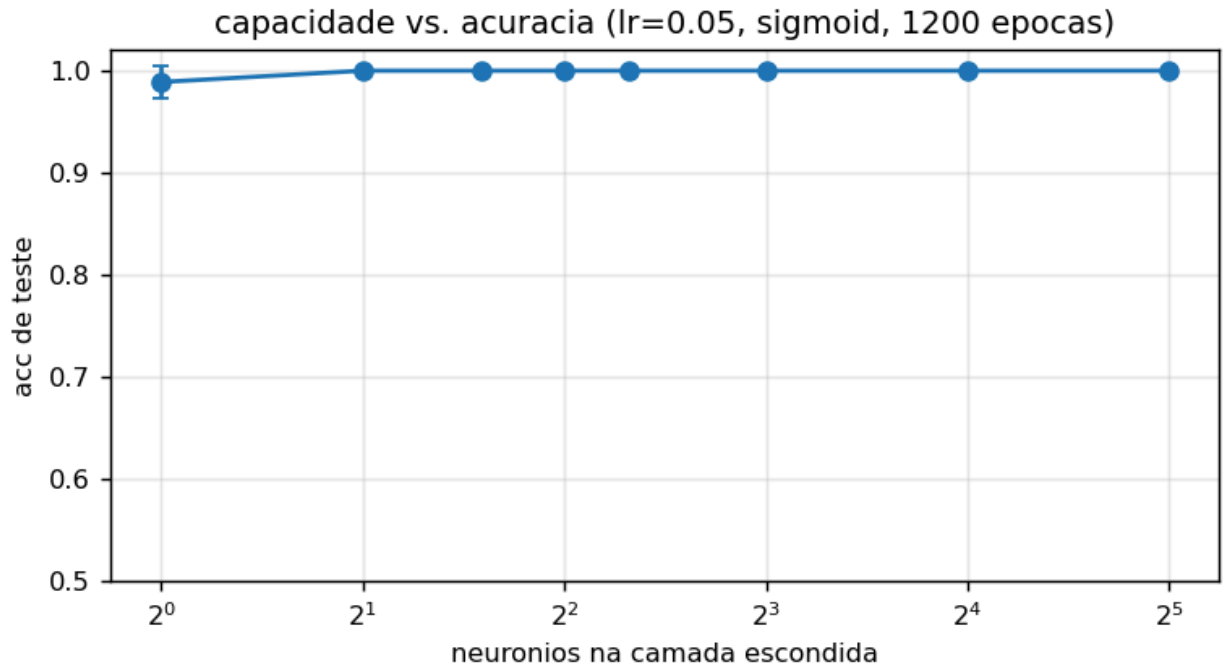


Figura 5: acuracia de teste vs. neuronios na camada escondida (media +- desvio, 3 seeds)

Ativação. Aqui veio a surpresa do dia (lr=0.05, H=5, 1200 épocas):

```
sigmoid (bias=-1)   acc=1.000 +- 0.000
tanh    (bias=-1)   acc=1.000 +- 0.000
relu    (bias=-1)   acc=0.322 +- 0.016
relu    (bias= 0)   acc=0.856 +- 0.140
```

ReLU com bias inicial -1 é uma rede morta: os pré-ativações nascem negativos, a saída dos neurônios trava em 0, o gradiente é 0 e nada aprende. A rede colapsa prevendo uma classe só (~33%). Subindo o bias pra 0 ela revive, mas continua instável (uma seed ficou em 0.67). ReLU pede uma inicialização melhor que "uniforme +-1 e bias constante". Pra essa arquitetura nesse dataset, sigmoid e tanh são as escolhas seguras, e é por isso que o notebook original usa sigmoid.

6. Overfitting

Com dados limpos não tem overfitting pra mostrar, e isso já é resultado. Treinando H=4 e H=64 por 2500 épocas, a validação acompanha o treino o tempo todo (val_acc bate 1.0 já na época ~25 e não sai de lá). Iris normalizado é separável demais: capacidade extra não degrada a generalização, só desperdiça época. Early stopping na melhor validação (~época 25-200) economiza ~90% do treino sem custar nada.

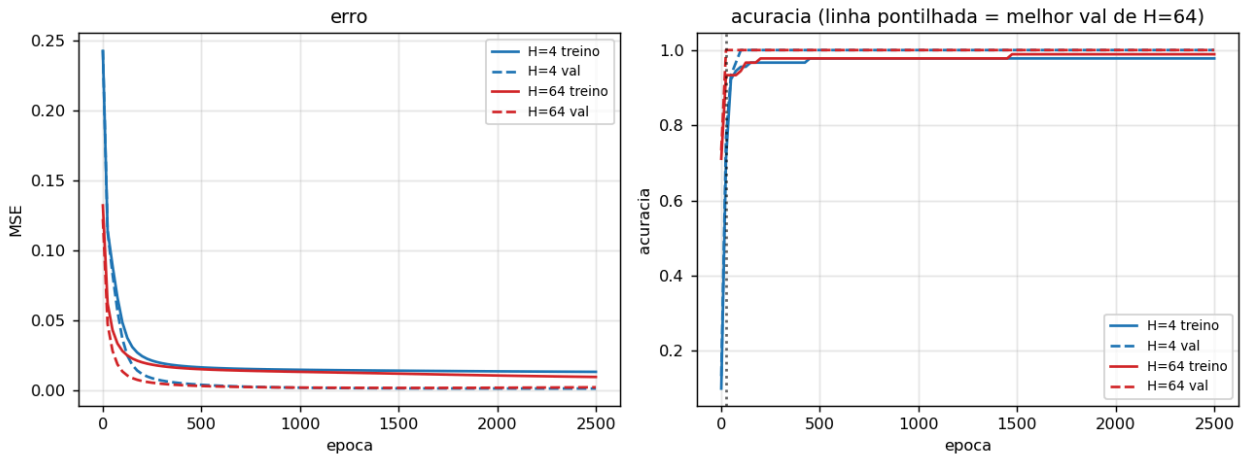


Figura 6: dados limpos: nem H=64 overfitta, val acompanha treino (Iris e separavel demais)

Pra ver o fenômeno de verdade eu injetei 20% de rótulos embaralhados no treino. Aí a assinatura clássica aparece no modelo grande: o H=64 vai memorizando o ruído, a acurácia de treino (medida contra os rótulos ruidosos) sobe até 0.92 enquanto a validação limpa cai de 1.00 pra 0.97, e o val_loss faz fundo ~época 1000 e volta a subir com o train_loss ainda caindo. O H=4, sem capacidade pra memorizar, segura a validação em 1.0. Resumindo: gap treino-val crescendo = overfitting; o remédio aqui é early stopping (melhor val na época 50) ou menos capacidade.

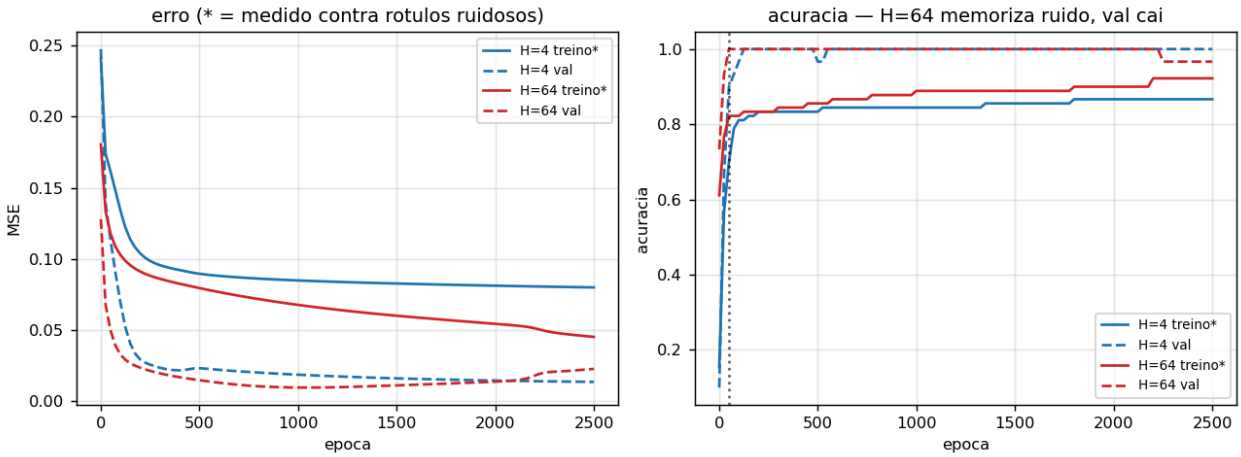


Figura 7: com 20% de ruido nos rotulos: H=64 memoriza o ruido (treino* sobe, val cai; val_loss diverge a partir de ~epoca 1000)

7. Como classificar bem as três classes

Juntando o que os experimentos mostraram, a receita ficou:

1. predict com ativação (o bug nº1; sem isso nada importa)
2. normalizar as features (z-score), destrava a sigmoid e encurta o treino umas 4x
3. sigmoid ou tanh; relu só com init e bias melhores
4. lr ~0.05 (0.005-2.0 funcionam, o 0.005 original é só lento)
5. H pequeno basta (H=2 já dá 1.0; usei H=8 por margem)
6. early stopping na validação: ~200-800 épocas, não 3000

Config final: `4 -> 8 -> 3`, sigmoid, lr=0.05, bias=-1, 800 épocas, dados normalizados -> 100% no teste (30/30), empatando com `sklearn.MLPClassifier(8)` que também dá 1.0. A matriz de confusão sai perfeita, nem versicolor vs. virginica escapam.

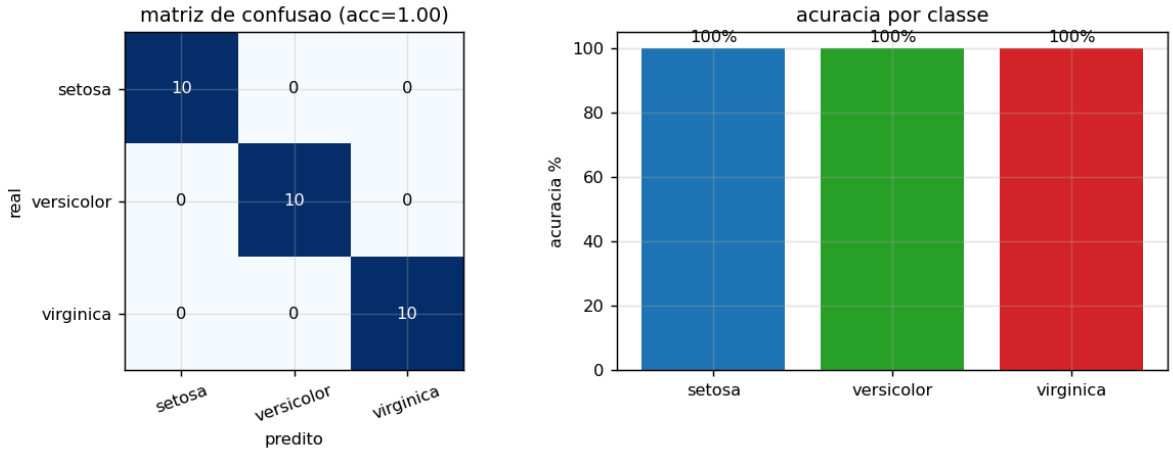


Figura 8: modelo final (4-8-3, sigmoid, lr=0.05): matriz de confusao e acuracia por classe

E como as pétalas carregam quase toda a informação, uma rede 2-8-3 treinada só em `petal length/width` também fecha 100% de teste. As regiões de decisão ficam quase lineares, com uma curva suave entre versicolor e virginica exatamente onde os pontos se encostam:

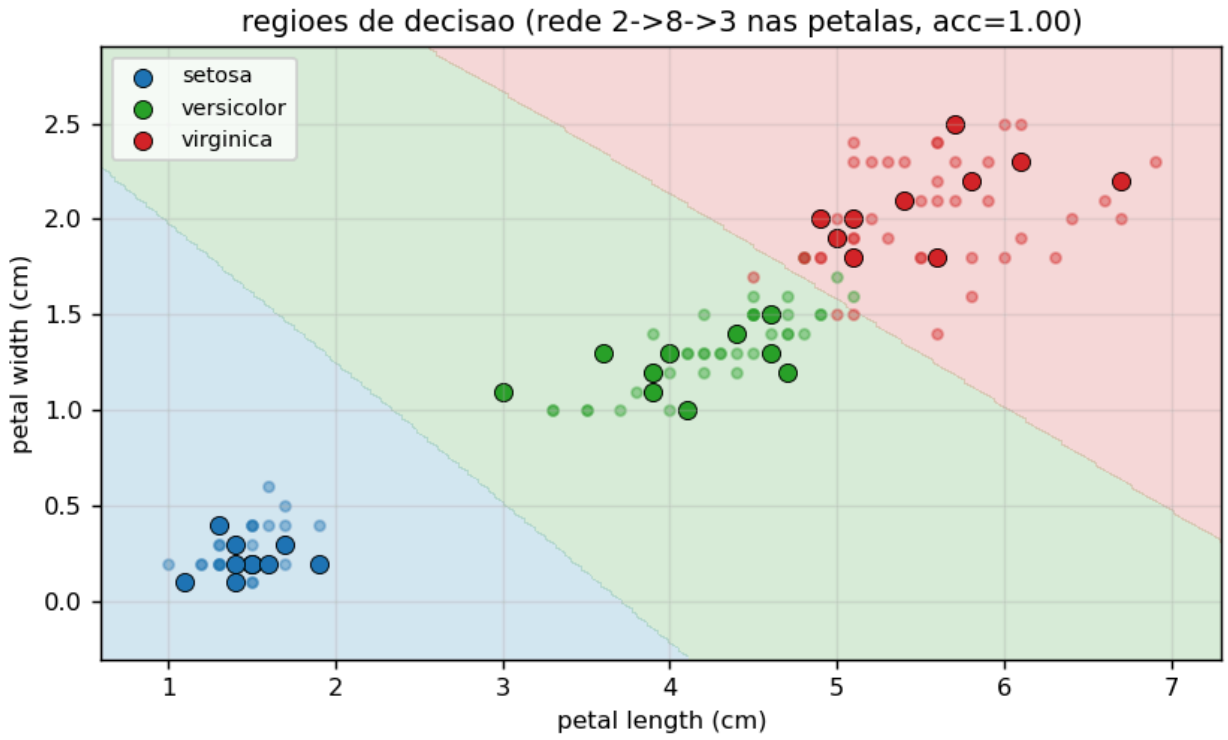


Figura 9: regioes de decisao de uma rede 2-8-3 treinada so nas features de petala (pontos grandes = teste, pequenos = treino)

8. Conclusão

A implementação de referência tinha a mecânica do backprop correta, os sinais de gradiente estão certos, mas cinco bugs em volta (predict linear, bias no loop errado, bias int truncado, métrica de erro furada, typo no construtor) faziam o notebook reportar uma convergência bonita pra um modelo que, na minha seed, errava 2/3 do teste. Depurado, o mesmo MLP com normalização e parada cedo acerta 100% das três classes. A dificuldade da tarefa estava muito mais no código que no problema. E o Iris só deixa ver overfitting quando a gente estraga os rótulos de propósito: aí o modelo grande memoriza o ruído na hora e a curva de validação denuncia.

Referências

- Notebook da disciplina: "Multi-Layer Perceptron from scratch" (adaptado de V. G. Lemos, Kaggle - link no Colab do AVA)
- Slides da disciplina: "Multi-layer Perceptron (MLP)" (Prof. Eduardo Todt)
- scikit-learn docs: MLPClassifier, StandardScaler, load_iris